1 Pseudorandom Generators

The pioneering computer scientist John von Neumann said, "Anyone who considers arithmetical methods of producing random digits is, of course, in a state of sin." By this he meant that deterministic procedures cannot generate randomness from nothing: their outputs are completely determined by their inputs. More generally, a deterministic procedure cannot increase randomness in an information-theoretic sense (see the question below).

A *pseudorandom generator* sidesteps this limitation: it is an efficient deterministic procedure that transforms a random input into a longer string that *appears* uniformly random to any efficient procedure, in the sense of computational indistinguishability. It is formalized as follows.

Definition 1.1. A *deterministic* function $G : \{0,1\}^* \to \{0,1\}^*$ is a *pseudorandom generator* (PRG) with *expansion* $\ell(n) > n$ if:

- G can be evaluated by a polynomial-time algorithm,
- $|G(x)| = \ell(|x|) > |x|$ for all $x \in \{0,1\}^*$, and
- the ensemble $\{G(U_n)\}$ is pseudorandom, i.e., $\{G(U_n)\} \overset{c}{\approx} \{U_{\ell(n)}\}$.

The final property is very strong: by the composition lemma, it follows that $G(U_n)$ can be used in place of $U_{\ell(n)}$ in *any* efficient algorithm, with computationally indistinguishable results.

For any PRG G with expansion $\ell(n)$, the image $G(\{0,1\}^n)$ contains at most 2^n strings, a fraction at most $1/2^{\ell(n)-n}$ of all $\ell(n)$-bit strings. When, e.g., $\ell(n) \geq 2n$, this fraction is exponentially small, yet no PPT algorithm can distinguish (with non-negligible advantage) the output of G on a uniform seed from a uniformly random string.

Question 1. Prove (or at least sketch a proof) that a *statistically* pseudorandom generator cannot exist. *Hint:* think about the sizes of $G(\{0,1\}^n)$ and $\{0,1\}^{\ell(n)}$, and define an appropriate statistical test (i.e., subset of $\{0,1\}^{\ell(n)}$).

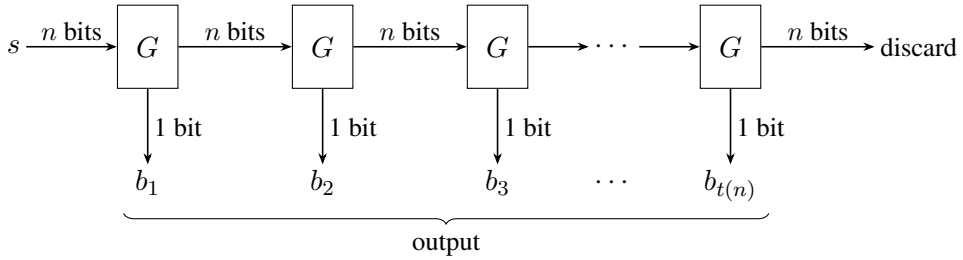
2 Expanding a PRG

The smallest possible PRG expansion is one bit: $\ell(n) = n + 1$. It turns out that this is sufficient to obtain *any* polynomial expansion!

Theorem 2.1. *Suppose that there exists a PRG G with expansion $\ell(n) = n + 1$. Then for any integer-valued $t(n) > n$ such that $1^{t(n)}$ is polynomial-time computable from 1^n (hence $t(n) = \mathrm{poly}(n)$), there exists a PRG G_t with expansion $t(n)$.*

Proof. We will construct $G_t(s)$ from G. Our construction applies G $t(n)$ times, outputting one new bit at each step and using the other n bits of the previous step's output as the next seed; see the following picture.

Formally, for any nonnegative integer k, let $G_k(s)$ denote the procedure from Algorithm 1, which produces k output bits from an n-bit seed s. The desired generator uses $k = t(n)$ on seeds of length n; we abbreviate $G_{t(n)}(s)$ as $G_t(s)$. By construction, its output has length $t(n)$. Since $t(n)$ is efficiently computable and polynomially bounded, and G is polynomial-time and called only on inputs of length n, G_T runs in polynomial time.



Algorithm 1 $G_k(s)$

if $k = 0$ **then**
  return ε (the empty string)
else
  let $(x|b) = G(s)$, where $x \in \{0,1\}^n$, $b \in \{0,1\}$
  return $b|G_{k-1}(x)$
end if

It remains to prove that $\{G_t(U_n)\} \overset{c}{\approx} \{U_{t(n)}\}$. Each invocation of G produces both an output bit and the seed for the next invocation, so these may be correlated. We must establish pseudorandomness of the entire output, accounting for these correlations. To prove this, we define a sequence of *hybrid* distributions going from the "real" output distribution $G_t(U_n)$ to the "ideal" uniform distribution $U_{t(n)}$. We then give a single algorithm that, given the index of an adjacent pair of hybrids, maps $G(U_n)$ to the earlier one and U_{n+1} to the later one. This algorithm will satisfy the conditions of the "polynomially many hybrid steps" corollary in Lecture 5, which combines the composition and hybrid lemmas to conclude that the first and last hybrid ensembles are computationally indistinguishable.

To give some intuition about how we design the hybrid experiments, we imagine that instead of invoking the first G on n uniform bits, what if we replaced its output with $n+1$ truly uniform bits? Intuitively, these two cases should not be distinguishable, because G is a PRG. And then what if we similarly replaced the first two invocations of G, and so on? Eventually, we would end up with $t = t(n)$ truly uniform output bits, as desired. Formally, the hybrid experiments are defined as follows:

- $H_0 = G_t(U_n)$,
- $H_1 = U_1|G_{t-1}(U_n)$,
- In general, $H_i = U_i|G_{t-i}(U_n)$ for $i \in \{0\} \cup [t]$,
- so $H_t = U_t$.

(Here $t = t(n)$, and we suppress the dependence of the hybrids on n.) All uniform strings in these expressions are sampled independently.

Define the algorithm $\mathcal{S}(1^n, i, z)$ below, and write $\mathcal{S}_i(1^n, z) := \mathcal{S}(1^n, i, z)$.

Algorithm 2 $\mathcal{S}(1^n, i, z)$ for $i \in [t(n)]$ and $z \in \{0,1\}^{n+1}$

parse z as $(x|b)$ for $x \in \{0,1\}^n$, $b \in \{0,1\}$
return $U_{i-1}|b|G_{t-i}(x)$

This algorithm runs in polynomial time uniformly over $i \in [t(n)]$, because it samples at most $t(n)$ bits

and makes at most $t(n)$ calls to G. If the input comes from $G(U_n)$, the recursive definition of G_k gives

$$\mathcal{S}_i(1^n, G(U_n)) \equiv U_{i-1} \| G_{t-i+1}(U_n) \equiv H_{i-1}.$$

If the input comes from U_{n+1}, then x and b are independent and uniform, so

$$\mathcal{S}_i(1^n, U_{n+1}) \equiv U_{i-1} \| U_1 \| G_{t-i}(U_n) \equiv U_i \| G_{t-i}(U_n) \equiv H_i.$$

These equivalences hold for every $i \in [t(n)]$. Since G is a PRG, the poly-many-hybrids corollary from Lecture 5 applies with $m(n) = t(n)$, $X_n = G(U_n)$, and $Y_n = U_{n+1}$, giving $\{G_t(U_n)\} \overset{c}{\approx} \{U_{t(n)}\}$, as required. □

To recap, our proof of Theorem 2.1 took the following path:

- We defined a construction: the candidate PRG G_t.
- We defined a sequence of hybrid experiments. In each step, we successively replaced *one* "real" invocation of a crypto primitive ($G(U_n)$) with its "ideal" counterpart (U_{n+1}).
- We gave a single algorithm $\mathcal{S}$ taking a hybrid index i as input, and verified that it maps $G(U_n)$ to the $(i-1)$st hybrid and U_{n+1} to the ith hybrid.
- This $\mathcal{S}$ is PPT, and $\{G(U_n)\} \overset{c}{\approx} \{U_{n+1}\}$ by hypothesis, so the polynomial-step corollary implies that the initial and final hybrid ensembles are computationally indistinguishable.

3 Obtaining a PRG

Thanks to Theorem 2.1, we know that all we need is to obtain a PRG with one extra bit of output. We describe a number-theoretic construction, due to Blum and Micali, of such an object.

3.1 Number Theory Background

Theorem 3.1 (Euler's theorem). *Let G be a finite abelian (i.e., commutative) multiplicative group. For every $a \in G$, we have $a^{|G|} = 1$.*

Proof. Consider the set $A = a \cdot G = \{ax \ : \ x \in G\}$. Because G is a group, a is invertible, and we have $A = G$. Taking products over all elements in $A = G$, we have

$$\prod_{x \in G} (ax) = \prod_{x \in G} x.$$

Because G is commutative, the LHS is $a^{|G|} \cdot \prod_{x \in G} x$, and we can multiply by the inverse of the RHS to obtain $a^{|G|} = 1$. □

When $G = \mathbb{Z}_p^*$ for a prime p, we have $|\mathbb{Z}_p^*| = \varphi(p) = p - 1$, so we obtain the following corollary:

Corollary 3.2 (Fermat's "little" theorem). *Let p be a prime. For any $a \in \mathbb{Z}_p^*$, we have $a^{p-1} = 1 \bmod p$.*

The following structural theorem will be very useful. (Its proof is elementary bur rather tedious, so we won't go through it today.)

Theorem 3.3. *Let p be a prime. The multiplicative group $\mathbb{Z}_p^*$ is* cyclic*, i.e., there exists some* generator *$g \in \mathbb{Z}_p^*$ such that $\mathbb{Z}_p^* = \langle g \rangle := \{g^1, g^2, \ldots, g^{p-1} = g^0 = 1\}$. (Equivalently, we can write the group as $\mathbb{Z}_p^* = \{g^0, g^1, \ldots, g^{p-2}\}$.)*

Question 2. Suppose there is a "black box" B that outputs a uniformly random element of $\mathbb{Z}_{p-1}$ for some known prime p. Show that there exists an algorithm that, using B as its only source of randomness, samples a uniformly random element of $\mathbb{Z}_p^*$.

3.2 Discrete Logarithm Problem and One-Way Function

Theorem 3.3 leads naturally to the so-called *discrete logarithm problem*, which is: given $y \in \mathbb{Z}_p^*$ (and prime p and generator g of $\mathbb{Z}_p^*$), find $\log_g y$, i.e., the $x \in \{1, \ldots, p-1\}$ for which $y = g^x \bmod p$. This problem is believed to be infeasible for large values of p.

Conjecture 3.4 (Discrete logarithm assumption). Let $\mathsf{S}(1^n)$ be a PPT algorithm that outputs some prime p and generator g of $\mathbb{Z}_p^*$. For every PPT algorithm $\mathcal{A}$,

$$\Pr_{(p,g)\leftarrow\mathsf{S}(1^n),y\leftarrow\mathbb{Z}_p^*}[\mathcal{A}(p,g,y) = \log_g y] = \mathrm{negl}(n).$$

We would like to design a collection of OWFs based on the discrete logarithm assumption. The collection is made up of the functions $f_{p,g} \colon \{1, \ldots, p-1\} \to \mathbb{Z}_p^*$ (for prime p and generator g of $\mathbb{Z}_p^*$), defined as

$$f_{p,g}(x) = g^x \bmod p.$$

Moreover, these functions are even *permutations* if we identify $\{1, \ldots, p-1\}$ with $\mathbb{Z}_p^*$ in the natural way.

It is a tautology that the collection is one-way under the discrete logarithm assumption. It is also clear that we can efficiently sample from the domain of $f_{p,g}$. But we still need to check that $f_{g,p}$ can be evaluated efficiently, and that (p, g) can be generated efficiently.

For the first, we use the standard "repeated squaring" technique for exponentiation, which requires $O(|x|)$ multiplications modulo p. The solution to the second issue is not entirely straightforward. Given only the prime p, it is unknown (in general) how to find a generator g of $\mathbb{Z}_p^*$ efficiently. However, given the *factorization* of $p-1$, which can be generated along with p, it is possible: every element in $\mathbb{Z}_p^*$ has order dividing $p-1$, so g is a generator if and only if $g^{(p-1)/q} \neq 1 \bmod p$ for every prime divisor q of $p-1$. The number of non-generators is at most the sum of $(p-1)/q$ over all prime divisors q of $p-1$, so the density of generators is typically large enough. An often-used special case is $p = 2q+1$ for prime q, for which there are $q = (p-1)/2$ generators. However, it is not even known whether there exist infinitely many such "Sophie Germain" primes of this form! (Empirically, though, they are abundant.)

3.3 Blum-Micali PRG

We now present a PRG that uses the ideas presented in the previous section. From Section 2 we know that if we have a PRG that is able to generate one extra bit of randomness, we can generate a polynomial number of pseudorandom bits. Our goal will be the following: we want to construct a PRG $G_{p,g} \colon \mathbb{Z}_p^* \to \mathbb{Z}_p^* \times \{0,1\}$.[1]

Our solution is a function with the following form:

$$G_{p,g}(x) = (f_{p,g}(x) = g^x \bmod p \quad , \quad h(x)).$$

Note that $f_{p,g}(x)$ performs the modular exponentiation function (which is a one-way under the discrete log assumption), while $h \colon \mathbb{Z}_p^* \to \{0,1\}$ is some function (yet to be defined) that provides the additional bit.

[1] To be pedantic, $G_{p,g}$ is a "collection" of PRGs, where the input seed comes from a set that depends on the function index (p, g). It it easy to check that our construction from Section 2 is compatible with this collection $G_{p,g}$.

Looking at the function, we can make the following observation: if $x \in \mathbb{Z}_p^*$ is chosen uniformly at random, then also $f_{p,g}(x)$ is uniform (because $f_{p,g}$ is a permutation). We still need to choose the function h, keeping in mind what we want from from the function: $h(x)$ should "look like a random bit," *even given* $f_{p,g}(x)$. That is, $h(x)$ should compute "something about x" that f hides *completely*.

We can think of many possible candidates for h: apply the xor function to all the bit of x; take the least significant bit of x (though you will show that this *does not* meet our requirements!); take the "most significant bit" of x (more precisely, test if $x > \frac{p-1}{2}$); etc.

Let's formalize the security property we want from h. Informally, we want it to be the case that no efficient algorithm, given $f(x)$, should be able to guess $h(x)$ with probability much better than the $\frac{1}{2}$ that is achievable by random guessing.

Definition 3.5 (Hard-core predicate). A predicate $h\colon \{0,1\}^* \to \{0,1\}$ is *hard-core* for f if for all PPT algorithms $\mathcal{A}$,

$$\mathbf{Adv}_{f,h}(\mathcal{A}) := \Pr_x[\mathcal{A}(f(x)) = h(x)] - \frac{1}{2} = \mathrm{negl}(n).$$

Next time, we will show that under the discrete log assumption, the "most significant bit" predicate $h(x) = [x > \frac{p-1}{2}]$ is hard-core for $f_{p,g}$.

Question 3. Prove that if h is hard-core for a one-way permutation $f\colon D \to D$, then

$$(f(x), h(x)) \overset{c}{\approx} (U(D), U_1) \equiv (f(x), b),$$

where $x \leftarrow D$ and $b \leftarrow \{0,1\}$ are independent. This means that $G(x) = (f(x), h(x)) \in D \times \{0,1\}$ is a PRG that expands by one bit. *Hint:* give a reduction that converts any distinguisher for the above two distributions into a predictor of $h(x)$ given $f(x)$.

Answers

Question 1. Prove (or at least sketch a proof) that a *statistically* pseudorandom generator cannot exist. *Hint:* think about the sizes of $G(\{0,1\}^n)$ and $\{0,1\}^{\ell(n)}$, and define an appropriate statistical test (i.e., subset of $\{0,1\}^{\ell(n)}$).

Answer. Since G has n-bit seeds (of which there are 2^n), there are at most 2^n possible outputs of G (which reside in $\{0,1\}^{\ell(n)}$). However, $\{0,1\}^{\ell(n)}$ is significantly larger than this; it has $2^{\ell(n)} \geq 2^{n+1}$ elements, making it at least twice as big as the image of G. Informally, we can think of our statistical test as checking whether or not its given string is in the image of G. When the string is an output of G, the check always succeeds; when the string is uniformly random, there is a significant (at least $1/2$) probability that the check fails.

More formally and in the language of statistical indistinguishably: let $A = G(\{0,1\}^n) \subseteq \{0,1\}^{\ell(n)}$, i.e., the image of G, as suggested above. Because G is deterministic, we have $|A| \leq 2^n$. Then

$$\begin{aligned} |G(U_n)(A) - U_{\ell(n)}(A)| &= |1 - U_{\ell(n)}(A)| \\ &= 1 - |A| \cdot 2^{-\ell(n)} \\ &\geq 1 - 2^{n-\ell(n)} \\ &\geq 1 - 2^{-1} = 1/2, \end{aligned}$$

which is non-negligible. Since we have demonstrated a particular A for which $|G(U_n)(A) - U_{\ell(n)}(A)| \geq 1/2$, the suprenum over all possible A cannot be smaller, so $\Delta(G(U_n), U_{\ell(n)}) \geq 1/2$.

Question 2. Suppose there is a "black box" B that outputs a uniformly random element of $\mathbb{Z}_{p-1}$ for some known prime p. Show that there exists an algorithm that, using B as its only source of randomness, samples a uniformly random element of $\mathbb{Z}_p^*$.

Answer. By Theorem 3.3, there exists a generator g of $\mathbb{Z}_p^*$. So, there exists an algorithm A that simply samples an $x \leftarrow \mathsf{B}$ and returns g^x. By Theorem 3.3, we know that $\langle g \rangle = \{g^0, \ldots, g^{p-2}\} = \mathbb{Z}_p^*$, and since $|\mathbb{Z}_p^*| = p - 1$, it must be the case that for each $x \in \mathbb{Z}_p^*$, there exists a *unique* $i \in \mathbb{Z}_{p-1}$ such that $x = g^i$. Hence, $\Pr[\mathsf{A} \text{ outputs } x] = \Pr[B \text{ outputs } i] = 1/(p-1)$, as needed.

Question 3. Prove that if h is hard-core for a one-way permutation $f \colon D \to D$, then

$$(f(x), h(x)) \overset{c}{\approx} (U(D), U_1) \equiv (f(x), b),$$

where $x \leftarrow D$ and $b \leftarrow \{0,1\}$ are independent. This means that $G(x) = (f(x), h(x)) \in D \times \{0,1\}$ is a PRG that expands by one bit. *Hint:* give a reduction that converts any distinguisher for the above two distributions into a predictor of $h(x)$ given $f(x)$.

Answer. We prove this by reduction. Let $\mathcal{D}$ be a PPT algorithm that seeks to distinguish $(f(x), h(x))$ from $(f(x), b)$, where $x \leftarrow D, b \leftarrow \{0,1\}$ are independent. We construct a PPT predictor $\mathcal{A}$ against the hard-core predicate h for f, which uses $\mathcal{D}$ as a black box and has the same advantage. The key idea is that $\mathcal{A}$, given some $y = f(x)$ for unknown x, will simply guess a candidate value b for the hard-core bit $h(x)$, and invoke $\mathcal{D}(y, b)$ to get some indication of whether this guess is correct. Since $\mathcal{D}$ may not be a perfect distinguisher, its answer may not always be "correct," so it requires some care to analyze this reduction.

Formally, we define the predictor $\mathcal{A}$ to work as follows: given input $y \in D$, choose a uniformly random bit $b \leftarrow \{0,1\}$ and invoke $\mathcal{D}(y,b)$. If $\mathcal{D}$ accepts (i.e., outputs 1), then output b as the prediction for $h(f^{-1}(y))$; otherwise, output $\overline{b}$ as the prediction. Clearly, $\mathcal{A}$ is PPT.

We now relate $\mathcal{A}$'s advantage in the prediction game to $\mathcal{D}$'s advantage in the distinguishing game. First, observe that $\mathcal{A}$'s advantage is

$$\begin{aligned}
\mathbf{Adv}(\mathcal{A}) &= \Pr_{x \leftarrow D}[\mathcal{A}(f(x)) = h(x)] - 1/2 \\
&= \frac{1}{2}\left(\Pr_{x \leftarrow D}[\mathcal{D}(f(x), h(x)) = 1] + \Pr_{x \leftarrow D}[\mathcal{D}(f(x), \overline{h(x)}) = 0] - 1 \right) \\
&= \frac{1}{2}\left(\Pr_{x \leftarrow D}[\mathcal{D}(f(x), h(x)) = 1] - \Pr_{x \leftarrow D}[\mathcal{D}(f(x), \overline{h(x)}) = 1] \right),
\end{aligned}$$

because $\mathcal{A}(f(x))$ outputs the correct bit $h(x)$ if $\mathcal{A}$ chooses $b = h(x)$ and $\mathcal{D}(f(x), b)$ accepts, or if $\mathcal{A}$ chooses $b = \overline{h(x)}$ and $\mathcal{D}(f(x), b)$ rejects. These events are disjoint, so we can add their probabilities, and b is uniform and independent of x, leading to the $1/2$ factor.

Now let

$$\begin{aligned}
p_U &= \Pr_{x \leftarrow D, b \leftarrow \{0,1\}}[\mathcal{D}(f(x), b) = 1] \\
&= \frac{1}{2}\left(\Pr_{x \leftarrow D}[\mathcal{D}(f(x), h(x)) = 1] + \Pr_{x \leftarrow D}[\mathcal{D}(f(x), \overline{h(x)}) = 1] \right),
\end{aligned}$$

because each of $b = h(x)$ and $b = \overline{h(x)}$ occur with probability $1/2$. Similarly, define

$$p_G = \Pr_{x \leftarrow D}[\mathcal{D}(f(x), h(x)) = 1].$$

By definition, $\mathcal{D}$'s advantage is

$$\begin{aligned}
\mathbf{Adv}(\mathcal{D}) = p_G - p_U &= \frac{1}{2}\left(\Pr_{x \leftarrow D}[\mathcal{D}(f(x), h(x)) = 1] - \Pr_{x \leftarrow D}[\mathcal{D}(f(x), \overline{h(x)}) = 1] \right) \\
&= \mathbf{Adv}(\mathcal{A}).
\end{aligned}$$

Since $\mathcal{A}$'s advantage is negligible by the hypothesis that h is hard-core for f, so is $\mathcal{D}$'s advantage, as desired.